

No Provisioned Concurrency: Fast RDMA-codesigned Remote Fork for Serverless Computing

Autores: Xingda Wei, Fangming Lu, Tianxia Wang, etc.
Publicação: arXiv:2203.10225v3 [cs.OS]

Aluno: Artur Rocha Lapot (15011640)



Roteiro

1. **Contexto:** O Dilema da Computação Serverless
2. **O Problema:** Cold Start vs. Custo de Recursos
3. **Soluções Atuais** e Suas Limitações
4. **A Proposta:** MITOSIS - Fork Remoto com RDMA
5. **Como Funciona?** Arquitetura e Detalhes Técnicos
6. **Avaliação Experimental** e Resultados
7. **Conclusão**



Contexto: O Dilema da Computação Serverless

- **A Promessa:** Pague apenas pelo tempo de execução, com escalabilidade "infinita" e automática.
- **Problema do "Cold Start":**
 - Iniciar um novo contêiner para uma função é lento (>100 ms).
 - Essa latência pode ser maior que o tempo de execução da própria função.
- **A Solução do "Warm Start":**
 - Manter contêineres "quentes" (em cache) para reuso é rápido (<1 ms).
 - **Custo Elevado:** Exige provisionar e pagar por muitos recursos ociosos, especialmente para lidar com picos de carga imprevisíveis.

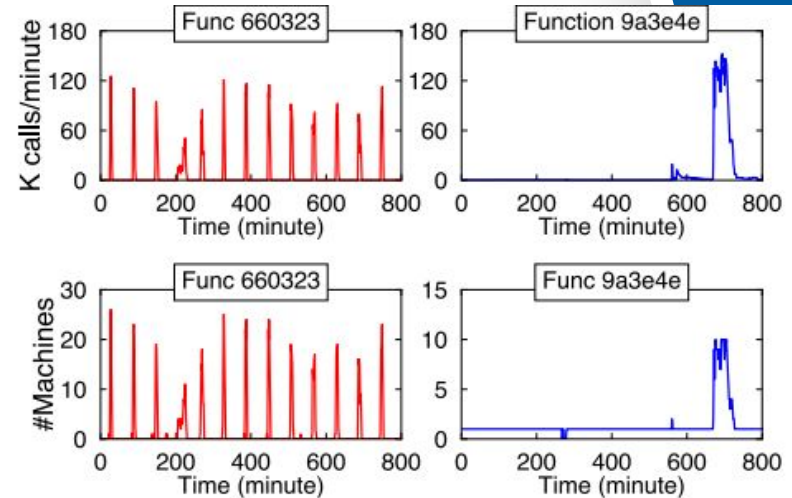


Figure 1. The timelines of call frequency (top) and sufficient resource provisioning (bottom) for two serverless functions in a real-world trace from Azure Functions [100].



Soluções Atuais e Suas Limitações

Técnica	Startup Remoto	Recursos Provisionados	Principal Limitação
Caching (Warm)	N/A	$O(n)$ - por função	Muito caro e consome muita memória.
Fork (Local)	N/A	$O(n)$ - por máquina	Não funciona em ambiente distribuído.
Checkpoint/Restore (C/R)	Lento (~24ms)	$O(1)$ - global	Muito lento devido a I/O e sobrecarga de software.

- Nenhuma solução atual oferece simultaneamente:
 - Startup remoto rápido.
 - Provisionamento mínimo de recursos ($O(1)$).



A Proposta: MITOSIS

Fork Remoto Rápido com Co-design de SO e RDMA

- **Conceito Central:** Estender a semântica do `fork()` para um ambiente distribuído. Criar um novo contêiner em uma máquina remota como se fosse um `fork()` local.
- **Como?** Através de um co-design profundo entre o kernel do SO e o hardware de rede **RDMA (Remote Direct Memory Access)**.
- Principais Benefícios:
 1. **Startup Eficiente:** Inicia contêineres remotos com a velocidade de um fork local, exigindo apenas um contêiner "semente" para todo o cluster.
 2. **Transferência de Estado Transparente:** O contêiner filho acessa a memória do pai diretamente (read-only), eliminando custos de serialização, cópias e armazenamento externo.

Como Funciona? C/R vs. MITOSIS



Checkpoint/Restore (Abordagem Tradicional)	MITOSIS (Nova Abordagem)
1. Pai: Faz checkpoint de toda a sua memória para arquivos.	1. Pai: Envia um descritor leve (~KB) com metadados para o filho.
2. Rede: Transfere esses arquivos (MBs/GBs).	2. Filho: Mapeia sua memória virtual para a memória física remota do pai.
3. Filho: Faz restore a partir dos arquivos.	3. Execução: Busca páginas de memória sob demanda via RDMA (one-sided).
Resultado: Lento, sobrecarga de software (DFS) e cópias de memória.	Resultado: Rápido, sem checkpoint de memória, sem cópias desnecessárias.

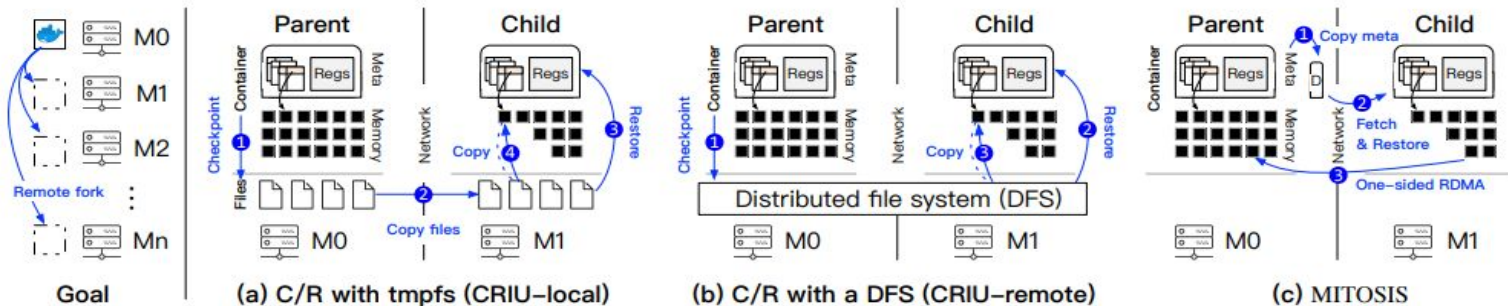


Figure 5. An overview of different approaches to achieve ultra-fast remote fork, including (a) C/R with a local filesystem (e.g., tmpfs), (b) C/R with a fast distributed filesystem (e.g., Ceph [5]), and (c) MITOSIS.

Arquitetura e Desafios Técnicos



- **Componentes Adicionados ao Kernel:**
 - **Fork Orchestrator:** Gerencia as chamadas `fork_prepare` e `fork_resume`.
 - **Network Daemon:** Gerencia conexões RDMA escaláveis (DCT).
 - **Gerenciador de Memória Virtual:** Modificado para tratar `page faults` remotas.
- **Desafios Técnicos Resolvidos:**
 1. **Conexão RDMA Escalável:** Uso de DCT (Dynamic Connected Transport) para evitar o alto custo de estabelecer conexões.
 2. **Controle de Acesso à Memória:** Um método de controle de permissão baseado na conexão (por VMA), sem o custo de registro de memória do RDMA.
 3. **Gerenciamento do Ciclo de Vida:** Delegado de forma inteligente para o coordenador da plataforma serverless.

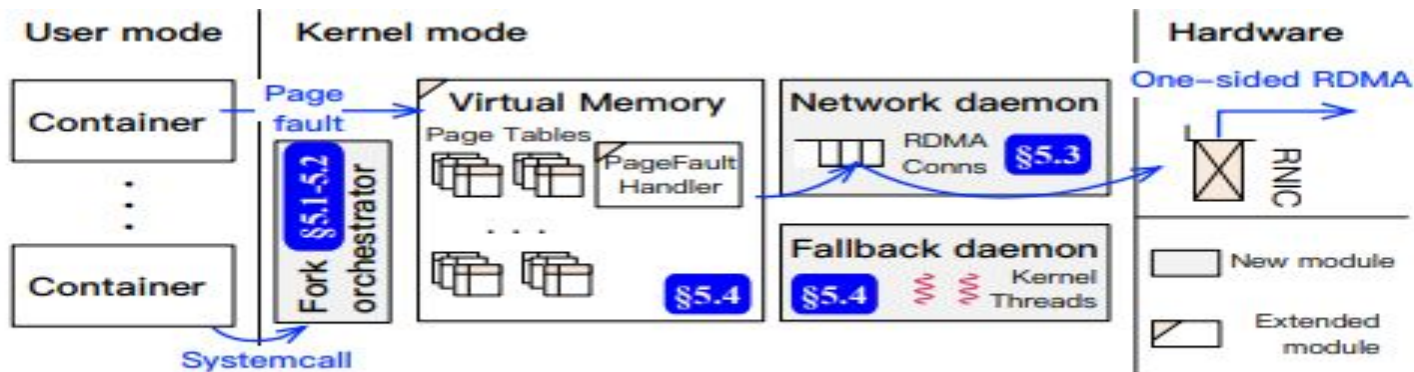


Figure 6. *The MITOSIS architecture.*



Avaliação e Resultados

- **Latência de Startup:**
 - MITOSIS inicia funções em **< 6 ms**.
 - Até **99% mais rápido** que C/R e **97% mais rápido** que o Cold Start.
- **Consumo de Memória:**
 - MITOSIS usa apenas **6.5%** da memória provisionada exigida por **Caching**.
- **Transferência de Estado (Aplicação FINRA):**
 - **86% mais rápido** que a abordagem padrão com Redis, eliminando a sobrecarga de serialização e rede.
- **Picos de Carga (Traces da Azure):**
 - Reduz a latência de cauda (p99) em **89%**.

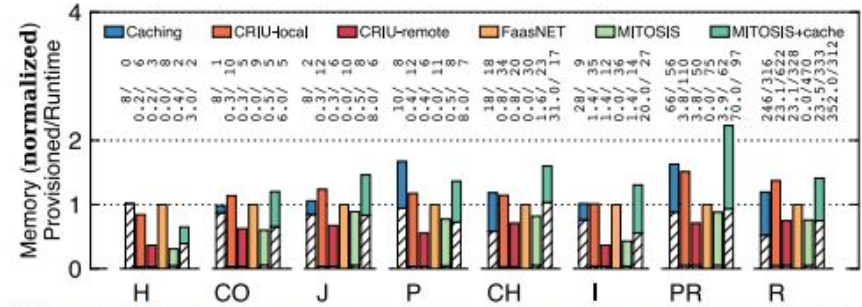


Figure 14. The per-function memory usage (in MB) for each technique before running (hatched) and during runtime (colored).

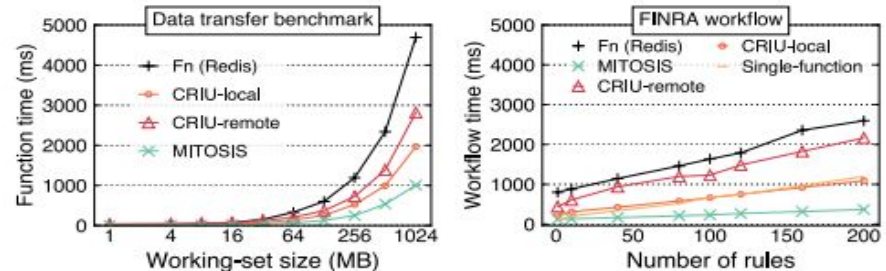


Figure 20. (a) The state-transfer performance between two functions and (b) performance of FINRA.



Conclusão

Problema: Plataformas serverless enfrentam um trade-off crítico entre **latência de startup** e **custo de recursos**.

Solução: **MITOSIS**, uma nova primitiva de SO, resolve esse dilema com um **fork remoto rápido**.

Inovação Chave: O **co-design entre o kernel do SO e o RDMA** permite acesso direto e sob demanda à memória remota, eliminando gargalos de software.

Impacto: MITOSIS torna a computação serverless mais rápida e barata, viabilizando startups quase instantâneos com provisionamento mínimo de recursos.